

UR3e_Motion_MTC — Motion Planning & Control Subsystem

Purpose

The Motion Planning and Control is responsible for all the physical movements of the UR3e robotic arm. This subsystem receives a brick's position within the pickup zone, and the target placement position on the LEGO board. Utilising MoveIt Task Constructor, the UR3e plans and executes the full pick-and-place sequence while avoiding collisions with itself and the surrounding environment.

System Requirements

Component	Version
Operating System	Ubuntu 22.04 (via WSL2 on Windows 11)
ROS2	Humble Hawksbill
MoveIt2	2.5.9
MoveIt Task Constructor	0.1.3
RViz2	11.2.26
WSL Version	WSL2 version 2.5.10.0
Robot	Universal Robots UR3e
Gripper	OnRobot RG2

Key Nodes, Topics, and Files

Node

Node	Description
<code>motion_node</code>	This ROS2 Node waits for pose data to be recieved from <code>ordered_pose_array</code> , then sets up the planning scene, planning and executing the pick-and-place task on the UR3e.

Subscribed Topics

Topic	Type	Description
<code>ordered_pose_array</code>	<code>std_msgs/msg/Float64MultiArray</code>	get placement position. See Inputs section for format. Consists of 12 float values representing the brick pickup and target placement positions. See Inputs section for format.

Key Source Files

File	Description
<code>src/ur3e_motion_mtc.cpp</code>	This is the main source file, containing the <code>MTCTaskNode</code> class, environment setup, pose subscriber, and <code>main()</code> .
<code>launch/ur3e_motion_mtc.launch.py</code>	This is the Launch File, loading the robot description, kinematics parameters, and planning configurations from <code>ur_onrobot_moveit_config</code> , and starts the <code>motion_node</code> .

External Dependencies

Package	Purpose
<code>rclcpp</code>	ROS2 client library for nodes, publishers, and subscribers
<code>moveit_core</code>	Contains core Moveit functionality
<code>moveit_task_constructor_core</code>	Contains core MTC planning pipeline
<code>std_msgs</code>	<code>Float64MultiArray</code> message type for pose input
<code>tf2_geometry_msgs</code> , <code>tf2_eigen</code>	Quaternion and transform utilities

Inputs and Outputs

Input — `ordered_pose_array`

A single `std_msgs/msg/Float64MultiArray` message containing 12 float64 values representing two poses in order:

```
[brick_x, brick_y, brick_z, brick_roll, brick_pitch, brick_yaw, target_x, target_y, target_z, target_roll, target_pitch, target_yaw]
```

Index	Value	Unit
0	brick x	metres
1	brick y	metres
2	brick z	metres
3	brick roll	radians
4	brick pitch	radians
5	brick yaw	radians
6	target x	metres
7	target y	metres

Index	Value	Unit
8	target z	metres
9	target roll	radians
10	target pitch	radians
11	target yaw	radians

All positions must be in the `world` frame, which within the current setup is equivalent to the `base_link`. The makes the coordinates relative to the base of the UR3e.

The node waits for the the `ordered_pose_array` to be received before planning and executing movements on the UR3e.

Output — Robot Motion

On receiving valid pose data the node executes this sequence on the UR3e:

1. Open gripper
2. Move arm to above-brick position (sampling planner)
3. Descend toward brick (Cartesian planner, downward -Z)
4. Close gripper around brick
5. Attach brick to gripper in scene
6. Lift brick upward (Cartesian planner, upward +Z)
7. Move arm to above-target position (sampling planner)
8. Lower to place position (Cartesian planner)
9. Open gripper
10. Detach brick from gripper in scene
11. Retreat upward (Cartesian planner, upward +Z)
12. Return arm to camera_home configuration (sampling planner)

How to Run

Building

```
cd ~/ws_moveit2
colcon build --symlink-install --packages-select ur3e_motion_mtc
source install/setup.bash
```

Launch Commands

Terminal 1 — UR driver (connect to physical robot):

```
source /opt/ros/humble/setup.bash
source install/setup.bash
```

```
ros2 launch ur_onrobot_control start_robot.launch.py \  
  ur_type:=ur3e \  
  onrobot_type:=rg2 \  
  robot_ip:=0.0.0.0 \  
  launch_rviz:=false
```

Terminal 1 — UR driver (connect to Fake Hardware):

```
source /opt/ros/humble/setup.bash  
source install/setup.bash  
ros2 launch ur_onrobot_control start_robot.launch.py ur_type:=ur3e  
onrobot_type:=rg2 use_fake_hardware:=true launch_rviz:=false
```

Terminal 2 — MoveIt + RViz:

```
source /opt/ros/humble/setup.bash  
source install/setup.bash  
ros2 launch ur_onrobot_moveit_config ur_onrobot_moveit.launch.py ur_type:=ur3e  
onrobot_type:=rg2 launch_rviz:=true
```

Terminal 3 — Launch Motion Planning and Control Node:

```
source /opt/ros/humble/setup.bash  
source install/setup.bash  
ros2 launch ur3e_motion_mtc ur3e_motion_mtc.launch.py
```

Terminal 4 — Test Pose Publish:

```
ros2 topic pub --once /ordered_pose_array std_msgs/msg/Float64MultiArray "{  
  data: [0.3, 0.3, 0.02, 0.0, 0.0, 0.0,  
         -0.25, 0.2, 0.03, 0.0, 0.0, 0.0]  
}"
```

Troubleshooting

If the MTC motion planning completes, but the execution fails, this may be an issue with the controllers.

Controller List:

First check which controllers are active and inactive.

```
ros2 control list_controllers
```

Activate Gripper Trajectory Controller:

```
ros2 control switch_controllers \
  --activate finger_width_trajectory_controller \
  --deactivate finger_width_controller
```

Activate Scaled Joint Trajectory Controller:

```
ros2 control switch_controllers \
  --activate scaled_joint_trajectory_controller \
  --deactivate joint_trajectory_controller
```

Opening MoveIt Task Constructor in RViz

On the bottom left window, click the **Add** button.

Under `moveit_task_constructor_visualization`, select **Motion Planning Task**.

In **Display** under **Motion Planning Task**, change **Task Solution Topic** to `/solution`.

Configurable Parameters

All parameters are set directly in `src/ur3e_motion_mtc.cpp`:

Parameter	Location in code	Default	Description
Approach min/max distance	<code>approach object</code> stage	<code>0.0 / 0.20 m</code>	Vertical descent range before grasping
Lift min/max distance	<code>lift object</code> stage	<code>0.0 / 0.20 m</code>	Vertical lift range after grasping
Retreat min/max distance	<code>retreat</code> stage	<code>0.0 / 0.20 m</code>	Vertical retreat range after placing
<code>move to pick</code> timeout	<code>stage_move_to_pick</code>	<code>15.0 s</code>	Max planning time to reach above the brick
<code>move to place</code> timeout	<code>stage_move_to_place</code>	<code>15.0 s</code>	Max planning time to reach above the place position
Max IK solutions (grasp)	<code>ComputeIK</code> grasp wrapper	<code>16</code>	IK solutions explored for grasp pose
Max IK solutions (place)	<code>ComputeIK</code> place wrapper	<code>8</code>	IK solutions explored for place pose
<code>task_.plan()</code> count	<code>doTask()</code>	<code>10</code>	Number of full task solutions to find before executing

Parameter	Location in code	Default	Description
Velocity scaling	<code>cartesian_planner</code>	<code>0.5</code>	Fraction of max joint velocity for Cartesian stages
Acceleration scaling	<code>cartesian_planner</code>	<code>0.5</code>	Fraction of max joint acceleration for Cartesian stages

Known Limitations and Assumptions

Assumptions:

- The pose data received from `ordered_pose_array` is in the `world` frame of reference.
- The pose data received from `ordered_pose_array` is an array of 12 float values representing the brick's pose and target pose, 6 vales for each.
- The node will only receive a single pick-and-place task per motion, and will only receive a new one once returning to the `camera_home` robot configuration.
- Two brick's will be distances far enough appart that the gripper can pick and place each brick using the brick's long axis.
- All required controllers have been actived before launching the `motion_node`.

Known Limitations:

- If planning fails, the node logs an error and exits. It does not retry automatically with different parameters.
- The workspace does not model the evironment of the actually UR3e. There is no placement board or other bricks, so the robot doesn't avoid collision with the environment yet.
- Currently, the Cartesian movements only work if the minimum is set to 0m, so they dont retreat or approach the bricks.